AI-Based Prioritization of Cascaded Vulnerabilities in Software Supply Chains Using Software Bills of Materials

Jiratchaya Phinyodom[1,a)] Yuta Shimamoto[1] Toshihiro Yamauchi[2]

Abstract: Modern software depends on third-party components, allowing attacks to propagate through software dependencies. Current SBOM-based approaches can identify vulnerable components and generate attack chains, but they provide limited support for prioritizing these chains according to their exploitation risk. This research proposes an AI-based framework that integrates SBOM-based attack chain analysis with exploit prediction to support risk-based prioritization of cascaded vulnerabilities. The framework extracts vulnerability features from CVEs, applies feature engineering and data imbalance handling, and predicts the exploitation probability of each vulnerability, which is then used to prioritize attack chains. To identify the most suitable exploit prediction model, Hybrid Generalized Additive Model (Hybrid GAM), Logistic Regression, and eXtreme Gradient Boosting (XGBoost) are compared across five data imbalance handling techniques on a dataset of 49,972 recent vulnerabilities, of which 194 were labeled as exploited. Experimental results showed that XGBoost combined with Borderline-SMOTE achieved the most suitable performance, with a 99.8% accuracy and 73.0% F1-score.

Keywords: Software supply chain security, Software Bill of Materials (SBOM), Cascaded vulnerabilities, Vulnerability prioritization, Exploit prediction

1. Introduction

Modern software increasingly relies on third-party and open-source components to make software development more effective and reduce the cost of development. However, these dependencies also increase the attack surface of software and the risks of attacks on the software supply chain, as vulnerabilities can be introduced through the use of the program dependency link and affect downstream components [1]. Recent attacks against an individual vulnerability in the software supply chain can affect multiple software products [2] , highlighting the importance of discovering security risks in software dependencies.

Software Bills of Materials (SBOM) provide visibility into software components, vulnerabilities, and dependency relationships within the software supply chain [3]. By integrating SBOM with vulnerability databases like the National Vulnerability Database (NVD) [4], vulnerable components can be discovered and mitigated. However, as software systems develop in complexity, organizations typically have to manage thousands of vulnerabilities with limited security resources [5]. Therefore, it is impossible to address all the vulnerabilities, and an effective vulnerability prioritization is important to mitigate security risks and enhance software supply chain security.

This research aims to determine whether exploit prediction models can provide meaningful prioritization of cascaded vulnerabilities. Existing SBOM-based approaches can identify vulnerable components and construct cascaded vulnerabilities from software supply chain information [6]. However, vulnerabilities should not be considered only in isolation, as relationships among them can form cascaded vulnerabilities that may introduce additional security risks. Furthermore, these approaches do not provide a mechanism for prioritizing cascaded vulnerabilities according to their exploitation risk. This limitation is important because organizations may need to manage thousands of vulnerabilities, making it impractical to remediate every potentially exploitable vulnerability within a short period of time.

Building on existing research on cascaded vulnerability generation, this study proposes an approach for prioritizing cascaded vulnerabilities using exploit prediction models and data imbalance handling techniques. A key challenge in exploit prediction is class imbalance, as exploited vulnerabilities represent a much smaller proportion of vulnerability datasets than non-exploited vulnerabilities [7]. Therefore, the proposed approach uses data imbalance handling techniques to address this issue.

To evaluate the proposed approach and identify the most effective combination of an exploit prediction model and

[1] Graduate School of Environmental, Life, Natural Science and Technology, Okayama University
[2] Faculty of Environmental, Life, Natural Science and Technology, Okayama University
[a)] jiratchaya-phiny@s.okayama-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

an imbalance-handling technique, three exploit prediction models are adopted for the evaluation: Hybrid Generalized Additive Models (Hybrid GAM) [7], Logistic Regression, and eXtreme Gradient Boosting (XGBoost). Several imbalance-handling techniques are also evaluated, including Borderline-SMOTE [8], SMOTE [9], SMOTE-Tomek [10], ADASYN [11], and class weighting [12]. Different combinations of these prediction models and imbalance-handling techniques are evaluated and compared. Based on their exploit prediction performance, the most suitable combination was identified. The selected combination is then applied to the proposed framework to examine whether it can be used for their prioritization.

The main contributions of this study are summarized as follows:

- Create the framework of SBOM-based cascaded vulnerability analysis integrate with exploit prediction models.
- Evaluate the most suitable model by comparing the different exploit prediction models with different dataset imbalance methods.
- Investigate whether the framework can produce meaningful prioritization results and use prediction models for prioritizing cascaded vulnerabilities effectively

2. Background and Related Work

2.1 SBOM-Related Challenges in Software Supply Chains

The existing research in this area can be categorized into two primary categories, which include vulnerability analysis of software supply chain based on SBOM and vulnerability exploit prediction. The first area is to find vulnerable software components and understand how vulnerabilities are connected through software dependencies, and the following is to estimate the probability that specific vulnerabilities will be exploited. This study builds on both areas by applying exploit prediction to previously identified cascaded vulnerabilities for risk-based prioritization.

The Regulation (EU) 2024/2847 [13], known as the CRA, is a horizontal regulatory framework of the European Union (EU), which applies to hardware and software products. They regularly assess cybersecurity risk, manage vulnerabilities, report third-party component vulnerabilities. In particular, the CRA requires organizations to provide SBOM in machine-readable format for vulnerability management. Software components and dependencies are revealed. Additionally, CRA enhances software vulnerability management and helps in the deployment of SBOM. The CRA also requires manufacturers to identify software components and manage vulnerabilities during the product lifetime. These requirements encourage the use of SBOM for software vulnerability management. Nevertheless, it is still difficult to guarantee the completeness and accuracy of the SBOM information and to find the most critical vulnerabilities to be fixed first.

2.2 SBOM-Based Vulnerability and Cascaded Vulnerability Analysis

Cascaded vulnerability attacks analyze dependency relationships and build vulnerability chains to extend SBOM-based vulnerability identification. The framework finds components and Common Vulnerabilities and Exposures (CVEs) in these chains and enriches analysis results to describe cascaded vulnerabilities. These outputs show how related software components can have multiple vulnerabilities. The current cascaded Vulnerability attacks framework focuses on identifying and building cascaded vulnerabilities [6]. However, the highest improvement priority for cascaded vulnerabilities based on exploitation risk is not determined.

2.3 Vulnerability Exploitation Prediction

Another related research area focuses on predicting whether known vulnerabilities are likely to be exploited. Traditional vulnerability assessment frequently relies on the Common Vulnerability Scoring System (CVSS) , which provides information about vulnerability severity and technical characteristics [14]. However, vulnerability severity alone does not necessarily indicate whether a vulnerability will actually be exploited. For this reason, exploit prediction approaches integrate additional information, such as the Exploit Prediction Scoring System (EPSS) [15] , historical exploitation information, and vulnerability metadata.

The Hybrid Generalized Additive Model (Hybrid GAM) [7] approach applies AI to vulnerability exploit prediction using multiple vulnerability characteristics. The approach combines structured vulnerability information with exploitation-related features to estimate the probability that an individual CVEs will be exploited. The experiments that are used in this study follow this direction by integrating NVD metadata, EPSS information, and exploited vulnerability labels and by constructing additional numerical, categorical, textual, and interaction features.

Class imbalance is also an important issue in vulnerability exploit prediction because confirmed exploited vulnerabilities represent only a small proportion of publicly disclosed vulnerabilities [7]. To address this problem, this study evaluates several imbalance-handling methods, including class weighting, SMOTE, SMOTE-Tomek, Borderline-SMOTE, and ADASYN. Logistic Regression and eXtreme Gradient Boosting (XGBoost) are also evaluated to compare exploit prediction performance under different imbalance-handling strategies.

2.4 Challenges of Prioritizing Cascaded Vulnerabilities using Software Bill of Materials

Challenge 1: Existing SBOM-based approaches identify cascaded vulnerabilities, but determining the meaningful prioritization of cascaded vulnerabilities remains challenging.

Challenge 2: Exploited vulnerabilities represent a much

© 2026 Information Processing Society of Japan

smaller proportion of the dataset than non-exploited vulnerabilities, which makes exploit prediction difficult.

Challenge 3: Different prediction models and imbalance-handling methods may produce different performance, making it necessary to determine an appropriate combination for supporting cascaded vulnerability prioritization.

3. Proposed Framework

3.1 Framework Concept

The proposed framework consists of three key components, each of which directly addresses one of the challenges identified in Section 2.4. They are described below in the order of the processing flow of the framework.

Addressing Challenge 2: Imbalance-aware exploit prediction (Sections 3.3–3.5).
Vulnerability information from the NVD and EPSS is used to construct features for exploit prediction, while the CISA Known Exploited Vulnerabilities (KEV) Catalog [16] provides exploited and non-exploited labels for model training. Because exploited vulnerabilities represent a much smaller proportion of the dataset than non-exploited vulnerabilities [7], imbalance-handling methods are applied to the training data.

Addressing Challenge 3: Systematic combination selection (Section 3.5 and Section 4).
Multiple exploit prediction models are evaluated with different imbalance-handling methods, and the most suitable combination is selected. The selected model produces an exploitation probability for each CVE.

Addressing Challenge 1: Cascade-level risk prioritization (Section 3.6).
The predicted exploitation probabilities are associated with the SBOM-based cascaded vulnerabilities generated by the existing Cascaded Vulnerability Attacks framework [6]. The exploitation probabilities of the CVEs contained in each cascade are then averaged to compute a cascade-level exploitation probability, which is used together with vulnerability-level information, such as CVSS scores, EPSS scores, and cascade length, to support the prioritization of cascaded vulnerabilities.

Unlike the previous work that either identifies cascaded vulnerabilities or predicts the exploitation of individual CVEs, this work combines imbalance-handling methods and exploit prediction models to prioritize cascaded vulnerabilities [6], [7]. The integration provides a prioritization of the generated cascaded vulnerabilities, offering more information to help decide which vulnerability chains should be addressed first.

3.2 Overall Framework Architecture

Existing research can identify and generate cascaded vulnerabilities in software supply chains [6]. However, it does not provide a mechanism for prioritizing these cascaded vulnerabilities according to their exploitation risk. Therefore, this study uses the cascaded vulnerability data generated by the existing framework as input and extends the analysis by applying exploit prediction to support risk-based prioritization. However, exploit prediction involves a highly imbalanced dataset because only a small proportion of vulnerabilities are exploited. This class imbalance presents a challenge for accurately predicting exploited vulnerabilities.

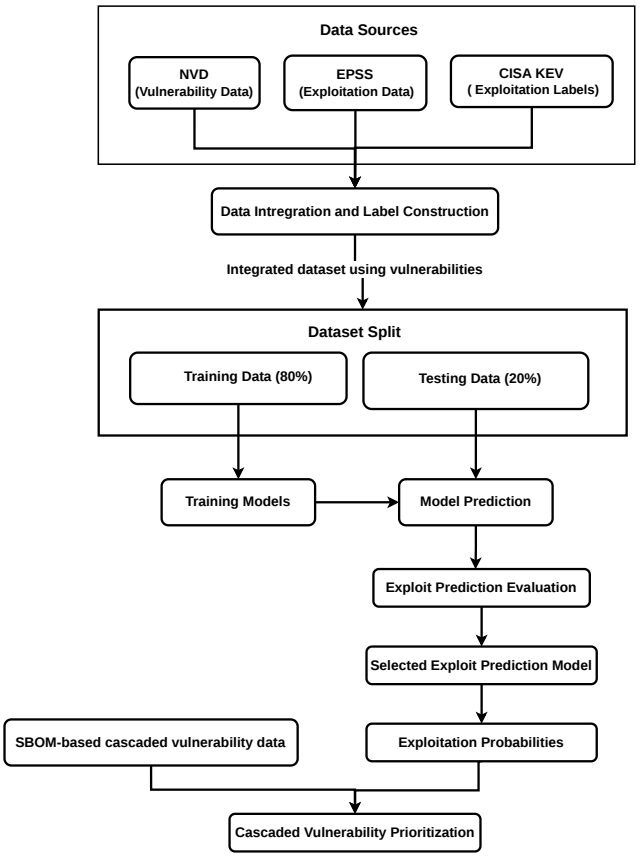


Fig. 1 Overall Framework Architecture

The overall framework architecture of this study is presented in **Fig. 1**, the framework uses vulnerability information from the NVD, exploitation likelihood information from EPSS, and known exploitation information from the CISA KEV Catalog. These data sources are integrated to prepare the vulnerability data for the subsequent processes. After data preparation the data will be integrated and ensure that they are normalized, consistent, and suitable for model training. Feature engineering is then performed to convert the vulnerability data into a feature matrix suitable for exploit prediction. Since the vulnerability dataset is imbalanced, different imbalance handling methods are applied with three exploit prediction models including Hybrid GAM, Logistic Regression, and XGBoost. The models and imbalance handling methods are compared to select the most suitable model. After model selection, the selected model is used to estimate exploitation probabilities. The estimated exploitation probabilities are then combined with the cascaded vulnerabilities generated by existing research to prioritize the cascaded vulnerabilities.

© 2026 Information Processing Society of Japan

3.3 Data Collection

3.3.1 Data Collection Overview

The data were obtained from the National Vulnerability Database (NVD) [4], the Exploit Prediction Scoring System (EPSS) [15], the CISA Known Exploited Vulnerabilities (KEV) Catalog [16], and JSON output results from the SBOM-based Cascaded Vulnerability Attacks framework [6]. The NVD metadata, EPSS scores, and CISA KEV labels were obtained from their publicly available datasets. The SBOM-based cascaded vulnerability data were obtained from the JSON outputs generated by the existing framework. The CVE identifier is used as a common key to integrate the datasets for feature engineering, exploit prediction, and cascaded vulnerability prioritization.

3.3.2 Vulnerability Data Sources and Integration

A key challenge in exploit prediction is obtaining the different types of information required for model training and evaluation. To address this challenge, this study integrates three data sources include NVD for vulnerability characteristics, EPSS for exploitation likelihood, and CISA KEV for generating the target labels.

The NVD dataset is collected from the publicly available website by requesting data through the official API [4]. The NVD provides a range of metadata for CVEs, including CVSS scores and vectors, exploitability scores, impact scores, vulnerability descriptions, publication dates, vulnerability status, Common Weakness Enumeration (CWE) classifications, and reference information. This metadata is used for feature engineering in the exploit prediction model.

The EPSS data set was taken from the official EPSS repository from FIRST [15]. The EPSS dataset comprises the EPSS score and EPSS percentile for each CVEs which is a probability for a vulnerability to be exploited in the wild. The CVEs identification connects the information with the NVD data. The EPSS score and percentile are used as numeric features. The EPSS score is used to create the engineering features `cvss_x_epss` and `age_x_epss` for the exploit prediction model.

The CISA KEV Catalog was collected from the official Cybersecurity and Infrastructure Security Agency (CISA) repository [16]. The CISA KEV Catalog contains vulnerabilities confirmed to have been actively exploited in the wild. In this study, the KEV Catalog is used to generate binary labels for model training. Vulnerabilities listed in the catalog are labeled as exploited, while the remaining vulnerabilities are labeled as non-exploited. The resulting dataset shows a class imbalance between exploited and non-exploited vulnerabilities.

3.3.3 SBOM-Based Cascaded Vulnerabilities

The SBOM-based cascaded vulnerability data from existing research [6] are used as the input for the cascaded vulnerability prioritization stage. The predicted exploitation probabilities are linked to the CVEs in the cascaded vulnerability data using the CVE identifier. By integrating NVD metadata with EPSS scores and classifying vulnerabilities using CISA KEV data, the framework builds a combined dataset for training the exploit prediction models. The exploit prediction results are then integrated with the SBOM-based cascaded vulnerability data for risk-based prioritization.

3.4 Data Integration and Preparation

3.4.1 Data Integration and Preparation Overview

NVD is used as the main source of CVE data. EPSS data are matched with the NVD data, while CISA KEV is used for exploitation information. The CVE identifier is used as the common key to connect the datasets. After data integration, the data are prepared to ensure that they are normalized, consistent, and suitable for model training. The integrated dataset is used to generate exploit labels and prepare the temporal training and testing datasets for the exploit prediction model. After exploit prediction, the exploitation probabilities are combined with the existing cascaded vulnerability data for cascaded vulnerability prioritization.

3.4.2 Data Preprocessing

The preprocessing of data procedure is similar to the general approach used in existing research, and the temporal data split is adapted for this study. The existing study uses vulnerabilities published in 2023 for training and vulnerabilities published in 2024 for testing [7]. In this study, the training data is comprised of the vulnerabilities published in 2025, and the testing data is comprised of the vulnerabilities published in 2026. The data are integrated, prepared, normalized, consistent, and ready for model training. The missing values are handled by feature types not listed in CISA KEV Catalog vulnerabilities are non-exploited. The textual description is converted using TF-IDF and SVD. Numerical features are continuous variables. The feature matrix is used for training the exploit prediction model.

CVE identifiers are normalized across datasets to create model training target labels. The CISA KEV Catalog generates positive CVE identifiers. Each integrated dataset vulnerability is compared to this positive set. If its CVE identifier is in the set, the vulnerability is labeled exploited (1); otherwise, non-exploited (0). We prepare training and testing datasets and evaluate exploit prediction models using these binary labels. This study splits vulnerabilities published from 2025 to 2026 by temporal train-test. The 2025 vulnerabilities are used for training, while the 2026 vulnerabilities are tested. Separated chronologically, previous vulnerability records can predict new vulnerabilities and prevent data leakage. The system checks the training and testing sets for exploited and non-exploited cases. A stratified random split is used if neither set has both classes. NVD and EPSS data are used for feature engineering. The features are numerical, categorical, textual, and engineered. Numerical features encode vulnerability severity, exploitability, and temporal properties, while categorical features encode CVSS vector vulnerability technical properties. TF-IDF and Truncated Singular Value Decomposition convert text to numbers. Combining numerical features to represent vulnerability severity, exploitation probability, and vulnerability age

© 2026 Information Processing Society of Japan

creates engineered features. These features are combined to represent vulnerability properties and feed exploit prediction models.

3.4.3 Numerical Vulnerability Features

The features used in this study for predicting exploits are CVSS metrics, EPSS scores, vulnerability age, and reference count, which are similar to the feature design in existing Hybrid GAM research. These features are incorporated into the proposed framework, and data imbalance handling methods are additionally applied to address the imbalance between exploited and non-exploited vulnerabilities.

Numerical vulnerability features include CVSS metrics, EPSS score and percentile, vulnerability age and reference count. CVSS metrics include base score, exploitability score and impact score, which describe the severity and technical features of each vulnerability. The EPSS score and percentile indicate the estimated probability of exploit. Vulnerability age is derived from the NVD publication date, while reference count is the number of external references associated with each CVE record. Missing numerical values are replaced with zeros to obtain a consistent feature representation.

3.5 Exploit Prediction Models

3.5.1 Exploit Prediction Models Concept

After feature engineering, the feature matrix is used to train the exploit prediction models. This study applies three exploit prediction models, which are Hybrid GAM, logistic regression, and XGBoost. Hybrid GAM is used since it has been employed for vulnerability exploitation prediction in previous studies. Logistic regression is used as a baseline classification model, while XGBoost is included as a tree-based boosting model. These models are different in terms of modeling approaches and are evaluated under the same experimental setting to compare their exploit prediction performance. Since the dataset has significantly less exploited vulnerabilities than non-exploited vulnerabilities, we combine the five different data imbalance handling methods including class weighting, ADASYN, SMOTE-Tomek, SMOTE, and Borderline-SMOTE. The selected model is then used to predict the exploit probability of each vulnerability in the vulnerability prioritization stage.

3.5.2 Imbalanced Learning Methods

To address the class imbalance problem between exploited and non-exploited vulnerabilities, this study tests several imbalanced learning strategies before training the exploit prediction models. By comparing the techniques with the selected exploit prediction models, the study evaluates which combination provides the most suitable exploit prediction performance for the imbalanced vulnerability dataset. The methods are applied only to the training data to improve the representation of the minority exploited class while preserving the original testing data. Each balancing technique is combined with the selected exploit prediction models to evaluate its effect on exploit prediction performance.

3.5.3 Threshold Optimization

Each exploit prediction model returns a probability score representing the likelihood that a vulnerability will be exploited. After model training, this study examines a range of decision thresholds using the predicted probabilities on the testing dataset. The optimal threshold is selected based on a weighted score combining F1-score, precision, and recall, while considering a minimum precision requirement. The selected threshold is then used to convert the predicted probabilities into exploited and non-exploited class labels.

3.5.4 Model Selection

Model selection is performed by comparing the predictions against reference exploitation labels obtained from the CISA KEV Catalog. The evaluation focuses on the capability to identify exploited vulnerabilities while maintaining a reliable prediction performance. The results are compared across the exploit prediction models and data imbalance handling methods to find the most suitable performing combination. The most suitable model is used to generate the exploitation probability for each vulnerability. The predicted exploitation probabilities are then used for cascaded vulnerability prioritization.

3.6 Cascaded Vulnerability Prioritization

3.6.1 Cascaded Vulnerability Prioritization Concept

After selecting the exploit prediction model, combined cascaded vulnerability data derived from SBOMs and the probability of exploitation estimated by the selected exploit prediction model are utilized to prioritize the cascaded vulnerabilities. The analysis starts with estimating the probability of exploitation of each vulnerability in the dataset. These probabilities are then mapped to the vulnerabilities that exist in each cascaded vulnerability chain. The vulnerability-level exploitation probabilities are averaged at the cascade level, and the resulting cascade exploitation probability is used to decide the priority ranking of the cascaded vulnerabilities.

3.6.2 Exploit Probability Estimation

The selected exploit prediction model is applied to the filtered dataset to estimate the probability that each vulnerability will be exploited. Rather than producing only binary exploited or non-exploited labels, the model outputs a probability score ranging from 0 to 1 for every CVE. These predicted exploitation probabilities are stored together with the corresponding vulnerability metadata and subsequently linked to the SBOM-based cascaded vulnerability data using the CVE identifier. This probability serves as the primary input for the vulnerability prioritization stage.

3.6.3 Cascaded Vulnerability Ranking

For each cascaded vulnerability, the associated CVEs are matched with their predicted exploitation probabilities. The predicted exploitation probabilities of the matched CVEs are then averaged to compute the cascade exploitation probability . The value is the average predicted exploitation probability for the vulnerabilities included in the cascade

© 2026 Information Processing Society of Japan

and is used as a basis for the cascaded vulnerability ranking. After the predicted exploitation probabilities of the cascaded vulnerabilities are calculated, the cascaded vulnerabilities are arranged in decreasing order of their average predicted exploitation probability. The cascade with the highest value is assigned the highest priority rank and the other cascades are ranked in a decreasing manner. The ranking provides prioritization of the detected cascaded vulnerabilities based on the predicted exploitation probability of the CVEs included in each cascade.

4. Exploit Prediction Evaluation

4.1 Experimental Setup

The research combines NVD, EPSS, and CISA KEV data. Vulnerabilities published in 2025 are used for training, while those published in 2026 are used for testing. The characteristics of these vulnerabilities are used as input features in exploit prediction models to determine the likelihood of their exploitation.

Before model training, the input features are standardized using `StandardScaler`, which is fitted on the training data and then applied to the test data to prevent data leakage. To resolve the imbalance between exploited and non-exploited vulnerabilities, five strategies are evaluated which are Borderline-SMOTE, SMOTE, SMOTE-Tomek, ADASYN and class weighting. For the resampling methods, the minority-to-majority sampling ratio is set to 0.20.The following describes each strategies.

Borderline-SMOTE: a targeted over-sampling strategy, it identifies minority class examples sitting near the class boundary and generates synthetic samples specifically around those, rather than uniformly across the whole minority class, leading to better detection of minority-class instances than standard SMOTE [8].

SMOTE: increases the representation of the exploited minority class by generating synthetic samples from existing minority-class instances [9].

SMOTE-Tomek: uses SMOTE to over-sample the minority class and then applies Tomek Links to remove overlapping or borderline majority-minority pairs, thereby cleaning up noisy or ambiguous samples near the decision boundary [10].

ADASYN: to use a weighted distribution for different minority class examples according to their level of difficulty in learning, where more synthetic data is generated for minority class examples that are harder to learn compared to those minority examples that are easier to learn [11].

class weighting: computing weights for each class rather than resampling the dataset so that the classifier is less biased toward the majority class while allowing a controlled tradeoff between true positive and false positive rates [12].

In this study we test three classification models include logistic regression, XGBoost and Hybrid GAM. The following describes each classification models.

Hybrid Generalized Additive Model (Hybrid GAM): The Hybrid Generalized Additive Model (Hybrid GAM) based on existing research on vulnerability exploitation prediction [7]. The model combines numerical vulnerability features, including CVSS, EPSS, vulnerability age, and reference count, with textual features extracted from vulnerability descriptions using TF-IDF and SVD. These features are used to estimate the probability that a vulnerability will be exploited.

Logistic Regression: Logistic Regression is used as a baseline classification model for exploit prediction [17]. The model predicts the probability that a vulnerability belongs to the exploited class based on the engineered feature set. Logistic Regression is trained using the same prepared training data and preprocessing pipeline as the other candidate models. After model training, threshold optimization is performed to determine the classification threshold. We train it with a maximum of 2,000 iterations. For XGBoost we use 300 estimators, a maximum tree depth of 4, a learning rate of 0.05, sub-sampling and column sampling ratios of 0.9, and class weighting based on the ratio between non-exploited and exploited samples. And for Hybrid GAM, we use LogisticGAM with linear terms for all features and apply grid search to optimize the model during training.

Extreme Gradient Boosting: Extreme Gradient Boosting (XGBoost) is implemented as one of the candidate exploit prediction models. XGBoost uses gradient-boosted decision trees to learn relationships among the vulnerability features and produces an exploitation probability for each vulnerability [18]. XGBoost is trained using the same prepared training data and evaluated using the same experimental procedure as the other candidate models. After model training, threshold optimization is performed to determine the classification threshold.

Each model computes an exploitation probability for the test vulnerabilities. Because the data is imbalanced, the research applies threshold optimization because the default threshold of 0.5 might miss exploited vulnerabilities in an imbalanced dataset. Choosing a more appropriate threshold can enhance recall and F1-score while retaining acceptable precision. We set a decision threshold ranging from 0.01 to 0.99 and analyze it in increments of 0.005.

4.2 Exploit Prediction Performance

The performance of exploit prediction is evaluated by comparing the model predictions with the reference exploitation labels obtained from the CISA KEV Catalog. The vulnerabilities included in the KEV Catalog are labeled as exploited (1), while the vulnerabilities not included in the catalog are considered as non-exploited (0). The performance of each model and imbalance handling method is evaluated using accuracy, precision, recall, F1-score, and PR-AUC, as shown in **Table 1**.

The optimized decision threshold is used to compare three exploit prediction models and five imbalance handling methods in Table 1. Experiments compare XGBoost, Logistic Regression, and Hybrid GAM using PyGAM LogisticGAM with Borderline-SMOTE, SMOTE-Tomek, ADASYN, and

© 2026 Information Processing Society of Japan

Table 1 Performance Comparison of Exploit Prediction Models

Model Type	Imbalance Method	Threshold	Accuracy	Precision	Recall	F1	PR-AUC
XGBoost	Borderline-SMOTE	0.800	0.998	0.771	0.692	0.730	0.751
XGBoost	SMOTE	0.840	0.998	0.750	0.692	0.720	0.792
XGBoost	SMOTE-Tomek	0.840	0.998	0.750	0.692	0.720	0.792
XGBoost	ADASYN	0.825	0.998	0.750	0.692	0.720	0.783
XGBoost	Class Weight	0.980	0.998	0.875	0.538	0.667	0.746
Logistic Regression	Borderline-SMOTE	0.990	0.997	0.593	0.410	0.485	0.522
Logistic Regression	SMOTE	0.990	0.997	0.575	0.590	0.582	0.577
Logistic Regression	SMOTE-Tomek	0.990	0.997	0.575	0.590	0.582	0.577
Logistic Regression	ADASYN	0.980	0.996	0.522	0.615	0.565	0.557
Logistic Regression	Class Weight	0.975	0.994	0.382	0.744	0.504	0.589
Hybrid GAM	Borderline-SMOTE	0.990	0.996	0.531	0.436	0.479	0.466
Hybrid GAM	SMOTE	0.990	0.996	0.535	0.590	0.561	0.571
Hybrid GAM	SMOTE-Tomek	0.990	0.996	0.535	0.590	0.561	0.571
Hybrid GAM	ADASYN	0.990	0.996	0.500	0.615	0.552	0.549
Hybrid GAM	Class Weight	0.965	0.994	0.354	0.718	0.475	0.591

class weighting. For each experiment, the optimized classification threshold is applied to predicted exploitation probabilities and compared to ground-truth labels. Accuracy, Precision, Recall, F1-score, and PR-AUC are used to select the most suitable model. Each model calculates the test set's vulnerability exploitation probability, and threshold optimization determines the classification threshold.

Because exploited vulnerabilities represent the minority class, accuracy alone is insufficient for evaluating model performance. Therefore, particular attention is given to precision, recall, F1-score, and PR-AUC. Precision is the proportion of the predicted exploited vulnerabilities that are actually exploited and recall is the proportion of the known exploited vulnerabilities that are correctly predicted. F1 is the harmonic mean of precision and recall. PR-AUC is helpful for imbalanced datasets, as it captures the precision-recall tradeoff at different decision thresholds .

4.3 Most Suitable Selection

The evaluated exploit prediction models and data imbalance handling methods were compared based on their prediction performance using Precision, Recall, F1-score, and PR-AUC. The XGBoost model combined with Borderline-SMOTE achieved the most suitable overall performance in the evaluated results at the optimal decision threshold of 0.800. The model achieved an Accuracy of 0.998, Precision of 0.771, Recall of 0.692, F1-score of 0.730 and PR-AUC of 0.751. Based on these results, this combination was selected as the final exploit prediction model. The selected model was then applied to the integrated vulnerability dataset to estimate the probability of exploitation of each CVE. These predicted probabilities were then associated with the CVEs contained in the SBOM-based cascaded vulnerability data and used as input to the cascaded vulnerability prioritization process.

5. Cascaded Vulnerability Prioritization Validation

5.1 Experimental Setup

To prioritize cascaded vulnerabilities, the exploitation probabilities predicted by the selected model are combined with SBOM-based cascaded vulnerability data obtained from existing research. The predicted probabilities are associated with the vulnerabilities in each cascade using their CVE identifiers. The resulting vulnerability and cascade information is then used as input to the cascaded vulnerability prioritization process.

5.2 Cascaded Vulnerability Prioritization Results

This research aims to investigate whether the framework can produce meaningful prioritization results and use prediction models for prioritizing cascaded vulnerabilities effectively. The most suitable exploit prediction model was selected based on accuracy, precision, recall, F1-score, and PR-AUC results, and the SBOM-based cascaded vulnerability data was linked to both these metrics and the estimated exploitation probability. **Table A·1** provides an example of the cascaded vulnerability prioritization results generated with the chosen exploit prediction model. The CVEs associated with each cascaded vulnerability are mapped to their predicted exploitation probabilities. These probabilities are averaged to calculate the cascade exploitation probability. The cascaded vulnerabilities are then ranked in descending order according to their cascade exploitation probabilities, with higher values receiving higher priority. The results demonstrate that exploit prediction can be applied to SBOM-based cascaded vulnerability data to produce exploitation-likelihood-based prioritization of the identified cascaded vulnerabilities.

6. Discussion

6.1 Findings

The evaluation results are used to select an exploit prediction model from the evaluated results. The selected model

© 2026 Information Processing Society of Japan

provides the estimated exploitation probability of each individual vulnerability. These probabilities are then used as input to support the risk-based prioritization of previously identified cascaded vulnerabilities.

The results also show the importance of considering class imbalance when predicting vulnerability exploits. Since exploited vulnerabilities represent only a small proportion of the dataset, different imbalance-handling methods produce different advantages between Precision and Recall. In addition, threshold optimization allows the classification decision to be adjusted according to the features of the imbalanced dataset rather than relying only on the default threshold of 0.5.

The exploit prediction results provide the exploitation probabilities required for the cascaded vulnerability prioritization stage. The probabilities generated by the selected model are intended to be associated with individual CVEs within the SBOM-based cascaded vulnerabilities and used as input for cascade-level risk assessment. This information will support the prioritization of previously identified cascaded vulnerabilities according to their estimated risk.

6.2 Limitations

Proposed method has several limitations. The exploit labels derive from the CISA KEV Catalog, which labels vulnerabilities listed as exploited and non-listed as non-exploited for model training and evaluation. The proposed prioritization approach cannot consider cascaded vulnerabilities that are not identified by the SBOM-based framework. The generated ranking is based on the average predicted exploitation probability of the vulnerabilities within each cascade and does not consider environment-specific factors such as system exposure, software configuration, and security controls. In addition, further evaluation is required to determine whether highly ranked cascaded vulnerabilities correspond to practically important vulnerabilities.

7. Conclusion and Future work

This study proposes prioritizing cascaded vulnerabilities in software supply chains using exploit prediction and SBOM-based information. The proposed method utilizes exploit prediction models to estimate vulnerability exploitation probability. At the optimal decision threshold of 0.800, the exploit prediction model XGBoost combined with Borderline-SMOTE performed most suitable in the evaluated results. The model achieved 0.998 accuracy, 0.771 precision, 0.692 recall, 0.730 F1-score, and 0.751 PR-AUC. However, the ranking's effectiveness over other prioritization criteria requires further evaluation since the cascaded prioritization does not represent a comprehensive assessment of security risk. At the cascaded vulnerability prioritization stage, these predicted probabilities can be linked to previously identified vulnerabilities. This approach enables risk-based prioritization of cascaded vulnerabilities for vulnerability correction decisions.

Future work will incorporate these factors and evaluate the prioritization approach on more software systems. Visualization will also be created to make prioritized cascaded vulnerabilities easier to understand. Interactive features such as filtering vulnerabilities by risk level, component or probability of exploitation may also be considered to aid in vulnerability analysis and vulnerability prioritization.

References

[1] Williams, L., Benedetti, G., Hamer, S. et al.: Research Directions in Software Supply Chain Security, *ACM Trans. Softw. Eng. Methodol.*, Vol. 34, No. 5, pp. 1–38 (2025).

[2] O'Donoghue, E., Reinhold, A. M. and Izurieta, C.: Assessing Security Risks of Software Supply Chains Using Software Bill of Materials, *Proc. SANER-C 2024*, pp. 134–140 (2024).

[3] Nocera, S., Di Penta, M., Francese, R. et al.: If It's Not SBOM, Then What? How Italian Practitioners Manage the Software Supply Chain, *Proc. ICSME 2024*, pp. 730–740 (2024).

[4] NVD: National Vulnerability Database (NVD), National Institute of Standards and Technology (online), available from ⟨https://nvd.nist.gov/⟩ (accessed 2026-08-19).

[5] de Smale, S., van Dijk, R., Bouwman, X. et al.: No One Drinks From the Firehose: How Organizations Filter and Prioritize Vulnerability Information, *Proc. IEEE S&P 2023*, pp. 1980–1996 (2023).

[6] Baird, L. and Moin, A.: Towards Predicting Multi-Vulnerability Attack Chains in Software Supply Chains from Software Bill of Materials Graphs, *Proc. FSE Companion 2026*, pp. 1336–1340 (2026).

[7] Issa, N., Gruska, D. and Ali, L.: A Hybrid GAM-Based Model for Predicting Vulnerability Exploitation, *Proc. CoopIS 2025*, Springer, pp. 499–508 (2026).

[8] Han, H., Wang, W.-Y. and Mao, B.-H.: Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning, *Proc. ICIC 2005*, Springer, pp. 878–887 (2005).

[9] Chawla, N. V., Bowyer, K. W., Hall, L. O. et al.: SMOTE: Synthetic Minority Over-sampling Technique, *J. Artif. Intell. Res.*, Vol. 16, pp. 321–357 (2002).

[10] Azhar, N. A., Mohd Pozi, M. S., Mohamed Din, A. et al.: An Investigation of SMOTE Based Methods for Imbalanced Datasets With Data Complexity Analysis, *IEEE Trans. Knowl. Data Eng.*, Vol. 35, No. 7, pp. 6651–6672 (2023).

[11] He, H., Bai, Y., Garcia, E. A. et al.: ADASYN: Adaptive synthetic sampling approach for imbalanced learning, *Proc. IJCNN 2008*, pp. 1322–1328 (2008).

[12] He, J. and Cheng, M. X.: Weighting Methods for Rare Event Identification From Imbalanced Datasets, *Frontiers in Big Data*, Vol. 4, p. 715320 (2021).

[13] European Parliament and Council of the European Union: Cyber Resilience Act (Regulation (EU) 2024/2847), EUR-Lex (online), available from ⟨https://eur-lex.europa.eu/eli/reg/2024/2847/oj/eng⟩ (accessed 2026-08-19).

[14] FIRST: Common Vulnerability Scoring System v3.1: Specification Document, Forum of Incident Response and Security Teams (online), available from ⟨https://www.first.org/cvss/v3.1/specification-document⟩ (accessed 2026-08-19).

[15] FIRST: Exploit Prediction Scoring System (EPSS): Get the Data, Forum of Incident Response and Security Teams (online), available from ⟨https://www.first.org/epss/data⟩ (accessed 2026-08-19).

[16] CISA: Known Exploited Vulnerabilities Catalog, Cybersecurity and Infrastructure Security Agency (online), available from ⟨https://www.cisa.gov/known-exploited-vulnerabilities-catalog⟩ (accessed 2026-08-19).

[17] Iannone, E., Sellitto, G., Iaccarino, E. et al.: Early and Realistic Exploitability Prediction of Just-Disclosed Software Vulnerabilities: How Reliable Can It Be?, *ACM Trans. Softw. Eng. Methodol.*, Vol. 33, No. 6, pp. 146:1–146:41 (2024).

[18] Budholiya, K., Shrivastava, S. K. and Sharma, V.: An optimized XGBoost based diagnostic system for effective prediction of heart disease, *J. King Saud Univ. Comput. Inf. Sci.*, Vol. 34, No. 7, pp. 4514–4523 (2022).

© 2026 Information Processing Society of Japan

Appendix

A.1 Cascaded Vulnerability Prioritization Results

Table A·1 Example of Cascaded Vulnerability Prioritization Results

Rank	Chain ID	Component	CVEs	Chain Length	Avg. Exploit Prob.
1	239	pkg:pypi/certifi@2024.2.2, ..., pkg:pypi/zipp@1.0.0	CVE-2018-25045, ...,CVE-2025-27556	28	0.01264
2	125	pkg:golang/golang.org/x/crypto@v0.0.0-20201012173705-84dcc777aaee, ..., pkg:pypi/starlette@0.20.0	CVE-2020-29652, ...,CVE-2025-22872	43	0.01217
3	44	pkg:golang/github.com/gin-gonic/gin@v1.3.0, ..., pkg:pypi/werkzeug@0.14.1	CVE-2018-17142, ..., CVE-2025-27516	76	0.00086
4	254	pkg:pypi/flask@2.1.2, ..., pkg:pypi/setuptools@62.3.2	CVE-2022-40897, ..., CVE-2025-27516	8	0.00084
5	210	pkg:composer/guzzlehttp/guzzle@7.4.2, ..., pkg:pypi/urllib3@1.26.9	CVE-2022-29248, ..., CVE-2025-27516	24	0.00084
6	250	pkg:pypi/jinja2@3.1.2, ..., pkg:pypi/urllib3@1.26.9	CVE-2022-31116, ..., CVE-2025-27516	12	0.00084
7	7	pkg:pypi/aiohttp@3.8.1, ..., pkg:pypi/zope@5.5.1	CVE-2022-40896, ..., GHSA-V8GR-M533-GHJ9	37	0.00084
8	199	pkg:pypi/jinja2@3.1.2, ..., pkg:pypi/setuptools@62.3.2	CVE-2022-40897, ..., CVE-2025-27516	8	0.00084
9	38	pkg:pypi/jinja2@3.1.2, ..., pkg:pypi/zipp@3.17.0	CVE-2024-22195, ..., GHSA-W235-7P84-XX57	10	0.00084
10	30	pkg:pypi/aiohttp@3.8.1, ..., pkg:pypi/tornado@6.1	CVE-2022-29191, ..., GHSA-XF83-Q765-XM6M	168	0.00084

© 2026 Information Processing Society of Japan